



Engine Implementation Documentation

[Engine B](#) · [Engine C](#) · [Engine D](#) · [Engine E](#) · [Impact Score](#) · [Badge System](#)

Project	StreetLight-PK — Civic Issue Reporting Platform
Scope	Backend AI Pipeline: Layers 2 – 5
Author	Engine Implementation Team
Date	March 2026
Audience	Development Team Members

Table of Contents

1. Pipeline Overview
2. Database Schema
 - users
 - user_profiles
 - reports
 - report_interactions
 - comments
 - verification_requests
 - verification_votes
3. Engine B — Fraud Detection (Layer 2)
 - Check 1: Impossible Travel
 - Check 2: Duplicate Report Detection
 - Check 3: Spam Pattern Detection
4. Engine C — Community Verification (Layer 3)
 - Step 1: Creating a Verification Request
 - Step 2: Voting
 - Step 3: Score Calculation
 - Step 4: Timeout / Expiry
5. Engine D — Trust History (Layer 4)
 - Account Age Score
 - Report History Score
 - Fraud History Score
 - Behaviour Score
6. Engine E — Final Confidence Score (Layer 5)
7. Impact Score System
8. Badge & Tier System

1. Pipeline Overview

How a civic report flows through the validation pipeline

Every civic report submitted through the mobile app passes through a sequential validation pipeline before it appears on the community feed. The pipeline is split into two phases:

Pre-Persistence Phase (Blocking)

These steps run before the report is saved. If any hard block is triggered, an HTTP 400 error is returned to the user and nothing is saved.

Layer	Engine	Purpose
Layer 0	Input Validator	Image quality — blur, brightness, resolution
Layer 1	AI Engine	Classify the civic issue and verify GPS via EXIF
Layer 2	Fraud Detector	Detect GPS spoofing, duplicates, and spam patterns

Post-Persistence Phase (Non-Blocking)

These steps run after the report is saved. The mobile user receives a success response first; these run in the background. Errors here are logged but never shown to the user.

Layer	Engine	Purpose
Layer 3	Community Verification	Create a community vote request; notify nearby users
Layer 4	Trust History	Score the submitter's trustworthiness
Layer 5	Final Score Calculator	Combine all scores into a single confidence metric

2. Database Schema

PostgreSQL via Supabase — every table relevant to the engine pipeline

Table: users

Core identity record for every registered citizen.

Column	Type	Nullable	Description
id	Integer	No	Primary key
first_name	String	No	User's first name
last_name	String	No	User's last name
cnic	String	No	Pakistani CNIC (unique, indexed)
email	String	No	Login email (unique, indexed)
hashed_password	String	No	bcrypt-hashed password
role	String(50)	No	citizen (default) or admin
reset_token	String	Yes	Temporary token for password reset flow
created_at	DateTime (TZ)	No	Account creation timestamp (UTC)

Why created_at matters: Engine D uses this to compute the Account Age sub-score. A brand-new account gets a low trust score; older accounts are more trusted. This is the most passive and tamper-resistant signal of legitimacy.

Table: user_profiles

Extended user data. Every user has exactly one profile (1:1 relationship). This holds all the civic engagement metrics.

Column	Type	Nullable	Description
profile_id	Integer	No	Primary key
user_id	Integer (FK)	No	Unique FK — one profile per user
location	String	No	City/area set during signup

Column	Type	Nullable	Description
profile_image_url	String	Yes	Cloudinary URL of profile photo
total_reported	Integer	No	Lifetime count of reports submitted
total_solved	Integer	No	Count of user's reports marked RESOLVED
impact_score	Float	No	Cumulative engagement score (default 0.0)
last_known_lat	Float	Yes	Most recent GPS latitude from the app
last_known_lng	Float	Yes	Most recent GPS longitude from the app
fcm_token	String(255)	Yes	Firestore device token for push notifications
fraud_flags	Integer	No	Count of fraud detections (default 0)
created_at	DateTime (TZ)	No	Profile creation timestamp
updated_at	DateTime (TZ)	No	Auto-updated on any change

Why last_known_lat/lng matters: Engine C needs to know which users are physically near a newly reported issue to request their verification. The coordinates are updated each time a user opens the verification screen — creating a last-seen location cache. Why fraud_flags matters: Engine D reads this to penalise users caught cheating. Each fraud flag reduces the Trust Score by 25 points.

Table: reports

The central table. Every submitted civic issue creates one row here. Fields are grouped by the layer that populates them.

Identity & Content

Column	Type	Nullable	Description
id	Integer	No	Primary key
user_id	Int (FK)	No	The citizen who submitted this report
title	String	No	Short headline of the issue
description	Text	No	Detailed description from the user
category	Enum	No	POTHOLE or TRASH (set by AI, not the user)

Column	Type	Nullable	Description
status	Enum	No	PENDING / VERIFIED / TODO / IN_PROGRESS / RESOLVED
image_url	String	Yes	Cloudinary CDN URL of the uploaded image

Location

Column	Type	Nullable	Description
location_address	String	No	Human-readable address string
location_city	String	Yes	City name (optional)
location_lat	Float	Yes	GPS latitude from the mobile device
location_lng	Float	Yes	GPS longitude from the mobile device

Engagement Counters

Column	Type	Nullable	Description
support_count	Integer	No	Number of users who tapped Support (default 0)
verify_count	Integer	No	Number of users who tapped Verify (default 0)
views	Integer	No	Number of times this report appeared in a feed

Layer 2: Fraud Detection Results

Column	Type	Nullable	Description
is_flagged_for_spam	Boolean	No	Soft flag — saved but queued for admin review
duplicate_of_id	Int (FK)	Yes	FK to the original report if near-duplicate

Layer 3: Community Verification Result

Column	Type	Nullable	Description
community_score	Float	Yes	Weighted community vote score 0–100 (NULL until closed)

Layer 4: Trust History Result

Column	Type	Nullable	Description
trust_score	Float	Yes	Engine D trust score at the moment of submission

Layer 5: Final Confidence Score

Column	Type	Nullable	Description
combined_score	Float	Yes	Weighted combination of AI + community + trust
verification_status	String(20)	Yes	AUTO_VERIFIED / REVIEW_NEEDED / REJECTED / PENDING

Table: report_interactions

Tracks which users have clicked Support or Verify on a report. Used to enforce one interaction per user per report.

Column	Type	Nullable	Description
id	Integer	No	Primary key
report_id	Int (FK)	No	The report being interacted with
user_id	Int (FK)	No	The user performing the action
interaction_type	Enum	No	SUPPORT or VERIFY
created_at	DateTime (TZ)	No	When the interaction occurred

Why this table exists: support_count and verify_count on reports are simple counters. This table answers 'has the current user already supported this report?' — a per-user query that a counter alone cannot answer.

Table: comments

Stores citizen comments on reports.

Column	Type	Nullable	Description
id	Integer	No	Primary key
report_id	Int (FK)	No	The report being commented on
user_id	Int (FK)	No	The commenter

Column	Type	Nullable	Description
text	Text	No	Comment body
created_at	DateTime (TZ)	No	When the comment was posted

Table: verification_requests

One row per report that enters the community vote process. Created immediately after a report is persisted (Layer 3).

Column	Type	Nullable	Description
id	Integer	No	Primary key
report_id	Int (FK)	No	Unique — one verification per report
status	Enum	No	PENDING, COMPLETED, or EXPIRED
radius_m	Float	No	Search radius in metres (default 500 m)
min_votes	Integer	No	Minimum votes to auto-close (default 3)
timeout_hours	Integer	No	Hours until expiry if threshold not met (default 48)
community_score	Float	Yes	Computed weighted score 0–100 (NULL while PENDING)
total_votes	Integer	No	Running count of all votes cast
yes_votes	Integer	No	Count of YES votes
no_votes	Integer	No	Count of NO votes
created_at	DateTime (TZ)	No	When the request was opened
completed_at	DateTime (TZ)	Yes	When the request was closed

Design note: thresholds (radius_m, min_votes, timeout_hours) are stored per-row rather than only in code. This means the system can support different thresholds for different report types or areas in the future without a schema change.

Table: verification_votes

One row per individual community vote. A UniqueConstraint on (request_id, user_id) enforces one vote per user per request at the database level.

Column	Type	Nullable	Description
id	Integer	No	Primary key
request_id	Int (FK)	No	The request being voted on
user_id	Int (FK)	No	The voter
vote	Enum	No	YES (issue is real) or NO (cannot confirm)
weight	Float	No	Vote weight derived from voter's impact_score
distance_m	Float	Yes	Physical distance in metres from voter to report
created_at	DateTime (TZ)	No	When the vote was cast

Why weight is stored: it is derived from the voter's impact score at the moment they vote. Because impact scores change over time, storing the weight creates an immutable audit record. Why distance_m is stored: currently informational, but enables future distance-weighted voting and accuracy analysis.

3. Engine B — Fraud Detection (Layer 2)

`backend/ai_layers/layer2_fraud_detection/fraud_engine.py`

Engine B runs after AI classification (Layer 1) and before uploading to Cloudinary. Fraudulent reports are caught and dropped before any storage cost is incurred. All three checks are purely database-driven using SQLAlchemy — no external APIs, no paid services. This keeps fraud detection fast, offline-capable, and free.

Check 1 — Impossible Travel (GPS Spoofing)

What it checks: Is it physically possible for this user to be at this location?

Logic:

- Look up the user's most recent previous report from the reports table.
- Compute the straight-line distance using the Haversine formula (great-circle distance).
- Compute the time elapsed between the two submissions in minutes.
- If distance exceeds 100 km AND time elapsed is less than 2 minutes — hard block.

Why 100 km / 2 minutes: These thresholds eliminate false positives from GPS accuracy noise (a few hundred metres at most). No legitimate scenario exists where a user is 100+ km away from their previous location within 2 minutes. If triggered: HTTP 400 is returned, no report saved, and `ImpactScoreManager.deduct_points(PENALTY_SPOOFING=50)` is called with `fraud_flags` incremented by 1.

Check 2 — Duplicate Report Detection

What it checks: Does an identical issue already exist at essentially the same location?

Two-Phase Logic (optimised for speed):

Phase 1 — Coarse SQL filter:

- Same IssueCategory (pothole vs. trash)
- `created_at` within the last 14 days
- `location_lat` between `new_lat ± 0.001°` (approx ±111 metres)
- `location_lng` between `new_lng ± 0.001°` (approx ±111 metres)

Phase 2 — Precise Haversine: For each candidate from Phase 1, compute the exact Haversine distance in Python. If any candidate is within 100 metres, it is flagged as a duplicate.

Why two phases: Running Haversine in Python on every report would be slow without PostGIS. The bounding-box filter narrows candidates to a small set; only those undergo precise calculation. Why 100 metres: Phone GPS accuracy is typically 10–50 metres in urban areas. A 30-metre threshold was too tight — the same pothole photographed from slightly different angles could have coordinates 40–60 metres apart. Why 14 days: Prevents a resolved issue from blocking re-submission if the problem recurs.

Check 3 — Spam Pattern Detection

What it checks: Is this user submitting reports at an abnormally high rate?

Logic:

- Count the user's reports submitted in the last 1 hour (SQL COUNT query).
- If the count exceeds 20, flag as spam.

Why soft flag instead of hard block: A burst of legitimate reports is possible during a natural disaster or major storm. Soft-flagging lets the system preserve data and give admins the final decision. The report is saved with `is_flagged_for_spam = True`. `ImpactScoreManager.deduct_points(PENALTY_SPAM=30)` is called and `fraud_flags` is incremented.

Check Priority and Short-Circuiting

The three checks run in order. If Check 1 or Check 2 triggers, the remaining checks are skipped — there is no point checking for duplicates if GPS coordinates were fabricated. This optimises database query load.

4. Engine C — Community Verification (Layer 3)

`backend/ai_layers/layer3_community_verification/community_engine.py`

Engine C runs immediately after a report is saved (post-persistence, non-blocking). AI classification alone cannot verify that a reported issue physically exists — community members who are geographically close are the most reliable ground-truth source.

Step 1 — Creating a Verification Request

- A `VerificationRequest` row is created with status `PENDING`, `min_votes=3`, `timeout_hours=48`, `radius_m=500`.
- The engine queries `user_profiles` to find users within a 500-metre radius, excluding the report's author (users should not verify their own reports — this was an explicit bug fix).
- Nearby users are found using the same two-phase bounding box + Haversine strategy as Engine B.
- (Planned) A Firebase push notification is sent to each nearby user's `fcm_token`. The FCM token infrastructure is ready but Firebase Admin SDK has not yet been configured.

Step 2 — Voting

Users within 2 km of a pending report see it in their Verification Screen (fetched via `GET /verification/pending?lat=&lng;=`). They cast a YES (the issue is real) or NO (there is no such issue) vote.

When a vote is cast (POST `/verification/{request_id}/vote`):

- Validation: Confirm the request is still `PENDING` and the user has not already voted (enforced in code and by `UniqueConstraint` in the database).
- Weight calculation: $\text{weight} = \max(1.0, 1.0 + \text{impact_score} / 100.0)$. A user with `impact_score=0` gets weight 1.0. A user with `impact_score=200` gets weight 3.0.
- Distance recording: The voter's GPS coordinates are sent with the vote. Distance from voter to report is stored on the `VerificationVote` row.
- Counter update: `total_votes`, `yes_votes`, and `no_votes` on `VerificationRequest` are incremented.
- Auto-finalisation check: If `total_votes >= min_votes (3)`, the score is automatically calculated and the request is closed.
- Impact score award: The voter earns +5 impact points for contributing a vote.

Step 3 — Score Calculation

When the minimum vote threshold is reached (or when a timed-out request expires), the community score is calculated:

$$\text{community_score} = (\text{sum_yes_weights} / \text{sum_all_weights}) \times 100$$

Example Calculation:

Voter	Impact Score	Weight	Vote
Voter A	100	2.0	YES
Voter B	0	1.0	YES
Voter C	50	1.5	NO

$\text{sum_yes} = 2.0 + 1.0 = 3.0$ | $\text{sum_all} = 2.0 + 1.0 + 1.5 = 4.5$ | $\text{score} = (3.0 / 4.5) \times 100 = 66.7$

Why weighted votes: A simple majority is easily gamed by creating multiple accounts. A weight tied to `impact_score` — which accumulates through legitimate, fraud-checked activity — makes vote manipulation significantly harder. New or fake accounts get minimum weight (1.0); established community members get higher weight.

Step 4 — Timeout / Expiry

If a `VerificationRequest` has been `PENDING` for 48 hours without reaching the 3-vote threshold, it is moved to `EXPIRED` status. This happens lazily — there is no background scheduler. The expiry check runs at the start of every call to `GET /verification/pending`. Stale requests are cleaned up the next time any user opens the verification screen.

- If the request expired with at least 1 vote — the partial score is calculated from available votes and stored.
- If the request expired with 0 votes — `community_score` is left as `NULL`.
- A `NULL` community score is handled gracefully by Engine E, which uses a fallback weighting mode.

4. Engine D — Trust History (Layer 4)

backend/ai_layers/layer4_trust_history/trust_engine.py

Engine D runs after a report is saved (post-persistence). The trust score is a snapshot of how trustworthy the submitter is at the time of submission — it is stored on the Report row and does not update retroactively. Not all reports come from equally reliable sources; Engine D quantifies this as a 0–100 score.

The Four Sub-Scores

Sub-Score	Weight	Source	What it measures
Account Age Score	20%	User.created_at	How long the account has existed
Report History Score	30%	UserProfile.total_reported	How active the user has been
Fraud History Score	30%	UserProfile.fraud_flags	Whether the user has been caught cheating
Behaviour Score	20%	Recent reports count	Whether the user is in a suspicious burst

```
trust_score = (age_score × 0.20) + (history_score × 0.30)
+ (fraud_score × 0.30) + (behaviour_score × 0.20)

is_trusted = trust_score >= 40
```

Account Age (20%)

```
min(100, (age_days / 30) × 100)
```

A brand-new account (0 days) scores 0. An account 30+ days old scores 100. Linear growth in between. Bots and fake accounts are typically created shortly before abuse begins. A 30-day minimum discourages throwaway accounts. This is passive — the user doesn't have to do anything to earn it.

Report History (30%)

```
min(100, 30 + (total_reported / 10) × 70)
```

A user with 0 reports starts at 30 — not 0. This baseline avoids penalising legitimate new users. A user with 10+ reports scores 100. A user who has submitted multiple reports demonstrates they understand the system and that their reports pass fraud checks.

Fraud History (30%)

```
max(0, 100 - (fraud_flags × 25))
```

0 flags = 100 | 1 flag = 75 | 2 flags = 50 | 4+ flags = 0. This is the heaviest-weighted sub-score — a user caught cheating is a direct reliability risk. The penalty is steep (25 points per flag) but not instant elimination. Repeated offences will bring the score to zero.

Behaviour Score (20%)

```
5+ reports in last 1 hour → 20 (suspicious) otherwise → 80 (normal)
```

Even if a user's history is clean, a sudden burst in a single hour indicates automated or malicious behaviour right now. The score is binary (20 or 80) because the concern is the sudden unexplained burst pattern, not volume itself.

Trust Threshold: A `trust_score` ≥ 40 is considered trusted (`is_trusted = True`). The threshold is intentionally permissive — a new, clean, moderately active user should pass. The score primarily informs Engine E's weighting; it is not an outright block.

6. Engine E — Final Confidence Score (Layer 5)

backend/ai_layers/layer5_final_score/score_calculator.py

Engine E runs after Engine D, still in the post-persistence phase. It produces a single, authoritative confidence number (0–100) for each report by combining evidence from three independent sources: the AI model, the community, and the submitter's history.

Input Scores

Input	Source	Fallback if Missing
ai_confidence	Layer 1 AI classification model	0.0
community_score	Layer 3 community voting outcome	NULL — handled separately
trust_score	Layer 4 trust history evaluation	0.0

Two Weighting Modes

Normal Mode (community score exists)

```
combined_score = (ai_confidence × 0.40) + (community_score × 0.30) +  
(trust_score × 0.30)
```

No-Community Mode (community_score is NULL)

```
combined_score = (ai_confidence × 0.55) + (trust_score × 0.45)
```

Why two modes: Community verification may not happen for 48 hours or may never happen if no nearby users are available. The report still needs a valid confidence score immediately. In the absence of community input, the AI's confidence is up-weighted from 40% to 55% and the trust score becomes more significant.

Decision Thresholds

Score Range	Status	Meaning
≥ 85	AUTO_VERIFIED	High confidence — all signals agree this is a real issue
60 – 84	REVIEW_NEEDED	Moderate confidence — a human reviewer (admin) should check

Score Range	Status	Meaning
< 60	REJECTED	Low confidence — likely invalid, duplicate, or fraudulent

Why 85 for AUTO_VERIFIED: Three independent signals (AI, community, trust) must all align strongly to reach 85. It is deliberately a high bar so that automatic approval without human review is reserved for truly clear-cut cases.

7. Impact Score System

backend/utls/impact_score.py

The Impact Score is a cumulative metric stored in `UserProfile.impact_score`. It grows through positive civic engagement and shrinks through fraud violations. It serves two purposes: (a) it is the primary driver of vote weight in Engine C, and (b) it is the basis of the Badge and Tier system visible to users. All score changes go through `ImpactScoreManager` — no code should modify `impact_score` or `fraud_flags` directly.

Point Awards (Positive)

Event	Points	When Triggered
Report submitted and passes all fraud checks	+10	After Layer 2 passes
Community verification vote cast	+5	On POST <code>/verification/{id}/vote</code>
User's report is marked RESOLVED	+20	On admin status update

Point Deductions (Negative)

Event	Penalty	Increments <code>fraud_flags</code> ?
GPS spoofing detected	-50	Yes
Spam pattern detected	-30	Yes
Duplicate report submitted	-10	Yes

Effect on Vote Weight (Engine C Connection)

The weight formula in Engine C is: $\text{weight} = \max(1.0, 1.0 + \text{impact_score} / 100.0)$

Impact Score	Vote Weight
0	1.0
50	1.5
100	2.0
200	3.0
500	6.0

A highly engaged user's YES vote can carry 6× the weight of a new user's vote. This creates a natural incentive: the more you contribute honestly, the more influence you have over community verification outcomes.

8. Badge & Tier System

`frontend/lib/screens/profile_screen.dart`

The badge system is entirely frontend-driven — it reads `impact_score` from the API and maps it to a badge display. No separate badge table exists in the database; the badge is computed on-the-fly from the current score. This keeps the backend simple and ensures the badge is always in sync with the actual score.

Tier 1 — Rookie (0 – 200)

Sub-Badge	Score Range	Icon	Theme
Street Scout	0 – 66	■	Just getting started
Road Ranger	67 – 133	■	Covering ground
Eagle Eye	134 – 200	■	Sharp and observant

Tier 2 — Community Hero (201 – 500)

Sub-Badge	Score Range	Icon	Theme
Civic Warrior	201 – 300	■	Fighting for the community
Street Sentinel	301 – 400	■	Standing guard
Problem Buster	401 – 500	■	Making sure issues get fixed

Tier 3 — Master Reformer (501 – 800)

Sub-Badge	Score Range	Icon	Theme
Urban Legend	501 – 600	■	Known across the city
Infrastructure Icon	601 – 700	■	Symbol of civic pride
Trust Titan	701 – 800	■	Trusted and respected

Tier 4 — StreetLight Paragon (801 – 1000)

Sub-Badge	Score Range	Icon	Theme
City Architect	801 – 866	■	Shaping the city's future
Beacon of Change	867 – 933	■	Inspiring the community
Grand Overseer	934 – 1000	■	The highest rank

Badge Progression on Profile Screen

- Badge chip — the current sub-badge name, emoji, and tier label shown as a tappable chip near the user's name.
- Impact Score card — the numeric score, a progress bar (score/1000), and points remaining until the next tier.
- Badge info sheet — tapping the chip opens a scrollable bottom sheet showing all 12 badges with locked/unlocked state and score ranges.

Why the Badge System is Frontend-Only: Computing and storing badge state in the database would create redundancy — the badge is fully deterministic from impact_score, so storing it separately would risk inconsistency. The frontend recomputes the badge on every profile load from the fresh impact_score value returned by the API.